Diviser pour mieux régner: une reconfiguration sémantique et modulaire de la pile réseau pour les réseaux sans fil contraints

Ahmad Mahmod[1] et Julien Montavont[1] et Thomas Noel[1]

[1]*ICube Laboratory (CNRS UMR 7357), University of Strasbourg, France*

Les objets communicants peuvent être mis à jour à distance via radio grâce à l'envoi d'images complètes du micrologiciel embarqué, permettant de corriger des vulnérabilités ou d'adapter les paramètres de la pile protocolaire afin d'optimiser les performances. Dans les réseaux sans fil contraints et à pertes (LLWNs), ces mises à jour traditionnelles entraînent de longs délais et la perte de tous les états réseaux à cause du faible débit, du fort taux de pertes et du redémarrage des objets. Dans cet article, nous proposons une architecture modulaire de la pile réseau basée sur des fonctions virtuelles, permettant de reconfigurer la pile de communication à une granularité très fine, tout en maintenant le fonctionnement continu du réseau. Notre approche a été validée par des expérimentations sur la plateforme IoT-LAB, démontrant que les mises à jour au niveau des fonctions virtuelles réduisent fortement le temps nécessaire pour effectuer la mise à jour des objets par rapport aux mises à jour par images complètes, tandis qu'un déploiement hybride du plan de contrôle améliore le compromis entre convergence et robustesse en restaurant les états réseau après mise à jour.

Mots-clefs : Low-power Lossy Wireless Networks, Programmable Networks, Lightweight Virtualization

1 Introduction

Low-Power and Lossy Wireless Networks (LLWNs) underpin many IoT applications, including environmental monitoring and industrial automation. LLWN devices are resource-constrained in energy, memory, and processing, leading to low-rate, range-limited multi-hop communication. These constraints have historically promoted monolithic stacks hardcoded into firmware, where even minor changes (e.g., tuning a parameter) require full-image updates. Such updates are costly because they are large relative to available bandwidth, risking congestion and failures, and they typically require a reboot that clears the network state and interrupts service. These challenges motivate modular designs and semantic updates.

LLWN functionality is commonly structured into three planes: a control plane that makes forwarding, scheduling, and policy decisions; a data plane that processes and forwards packets; and a management plane that adapts behavior to meet network objectives. Control can be deployed in a centralized model (e.g., SDN), where a controller provides global coordination but depends on a reliable backhaul link, or in a distributed model, where nodes decide locally but may converge slowly and remain sub-optimal due to limited visibility. Early LLWNs largely adopted distributed control (e.g. RPL [GK12]) because reliable backhaul links are difficult in lossy duty-cycled multi-hop networks. More recently, SDN approaches tailored to LLWNs have revived interest in centralized control [B$^+$18], but both models retain fundamental limitations, motivating hybrid deployments.

We propose a flexible LLWN architecture with a modular network stack that decomposes protocols into lightweight Virtual Functions (VFs), enabling selective, function-level updates that substantially reduce update size. The architecture further supports semantic updates by deciding *what* to update and *when*, improving update safety and limiting service disruption. It also provides a unified framework that supports distributed, centralized, and hybrid control-plane deployments. Our contributions are: (i) a modular stack enabling selective and semantic updates; (ii) an architecture that supports distributed and centralized control-plane deployments and introduces hybrid control to LLWNs; and (iii) a real-world evaluation on a testbed. A preprint providing a detailed description of the architecture and evaluation is available. †

†. https://hal.science/view/index/docid/5491068

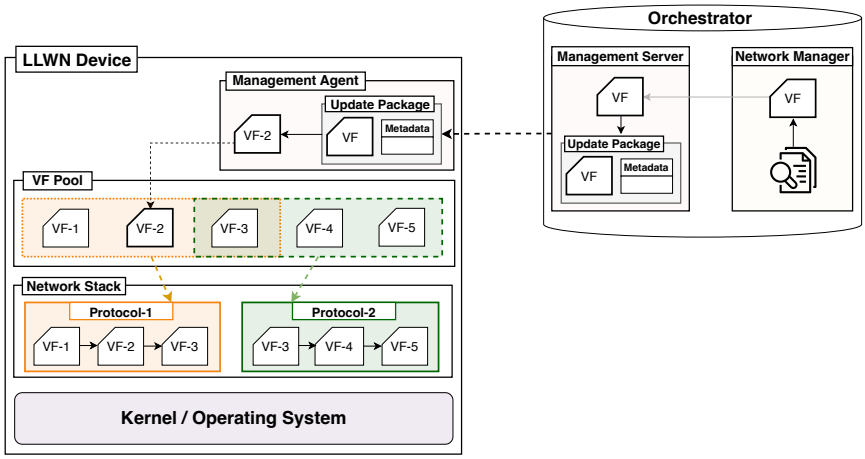


Figure 1 – Proposed Architecture

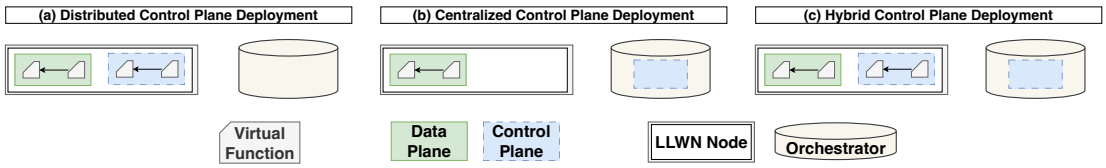


Figure 2 – Supported Control-Plane Deployments

2 Motivation

In distributed deployments, control and data planes are tightly coupled within each node, making even small modifications coarse-grained and disruptive. RPL illustrates this limitation; nodes construct the routing topology by exchanging control messages and operate in two modes. In storing mode, intermediate nodes maintain routing entries for their descendants; this state can exceed memory limits and may prevent some nodes from joining when no alternative parent is available. A common remedy is to switch to non-storing mode, where only the root maintains routing state and supports downward traffic through source routing [I$^+$20]. Today, such transitions typically rely on full firmware updates, which are costly over bandwidth-limited multi-hop links and can increase energy consumption, congestion, and the probability of failure. Full-image updates are also semantically blind: they usually require a reboot that discards routing state, triggers reconvergence through renewed control-message exchanges, and interrupts service. Centralized control can mitigate reconvergence by preserving and restoring state, but it depends on a reliable backhaul; congestion or disconnections can delay telemetry collection and policy dissemination. These limitations motivate a modular, semantics-aware network stack that supports selective runtime updates and enables hybrid control-plane deployments combining global optimization with local autonomy.

3 Proposed Architecture

The architecture couples a fine-grained, VF-based network stack with a central orchestrator to enable *semantic* runtime reconfiguration in LLWNs and support different control-plane deployments. The network stack is a chain of lightweight VFs, each implementing a minimal function (e.g., parsing, processing, state update, and rule installation). VFs are executed in isolated environments with low overhead, enabling safe updates and limiting fault propagation. This abstraction supports multiple control-plane realizations. In *distributed* deployments, VFs implement the local control- and data-plane protocol logic. In *centralized* deployments, VFs can encapsulate controller decisions as rules that are transmitted to install them on node data planes (SDN-like operation). In *hybrid* deployments, both coexist.

			Size (Byte)	Update Time (s)
Our architecture	***Centralized***		$N \times i; i \in \{104, 208\}$	364.95
	Distributed	*storing*	3,758	1828.17
		non-storing	2,966	1612.59
	Hybrid	*storing*	$3{,}758 + N \times i; i \in \{104, 208\}$	2170.23
		non-storing	$2{,}966 + N \times i; i \in \{104, 208\}$	1905.06
Native	***Distributed***	*storing*	123,000	27,041.04
		non-storing	125,728	27,376.47

Table 1 – Update Size and Time among Control-Plane Deployments

Fig. 1 details the architecture. Each node maintains a **VF Pool** that stores locally available VFs and enables reuse across layers and protocols. A central orchestrator configures the network over the air through two components: (i) a **Network Manager**, which supervises the network stack across devices—spanning the data and control planes—to coordinate configuration, scheduling, and adaptation. In addition to orchestration, it can host the control plane in some deployments when global optimization is required, improving performance and policy consistency; and (ii) a **Management Server**, which packages and delivers updates. Updates are transmitted as *Update Packages* that encapsulate a VF with metadata for authentication, integrity verification, versioning, and installation parameters (target platform, placement within the stack, and size). On devices, a **Management Agent** verifies the package, extracts the VF, and installs it into the VF Pool according to the metadata, enabling safe and semantically consistent reconfiguration.

Our architecture supports three deployment models as shown in Fig.2. In the **distributed** model (e.g., RPL protocol), control and data planes co-reside on each node and are implemented in VFs; protocol behavior is updated by replacing only the affected VFs instead of reinstalling the full image. In the **centralized** model (e.g., SDN), the orchestrator hosts the control plane and programs nodes by disseminating rule-carrying VFs, enabling cross-layer updates. The **hybrid** model combines both: nodes retain a local VF-based control plane for resilience under intermittent connectivity, while the orchestrator injects global decisions and, when needed, state information to accelerate post-update stabilization.

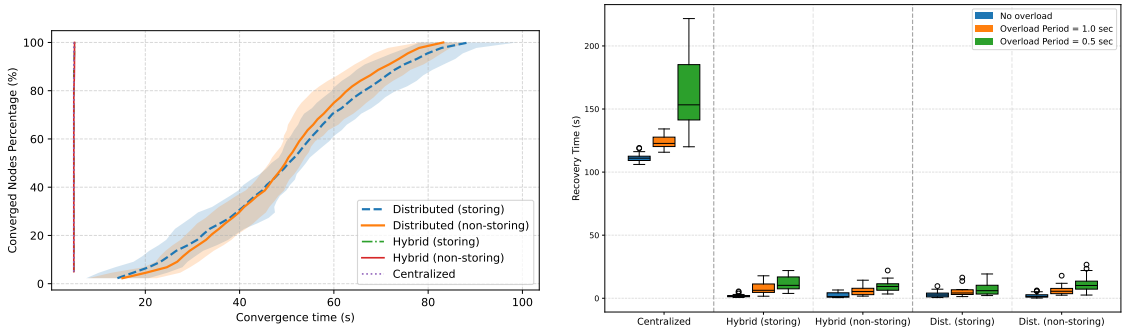
4 Implementation and Evaluation

We implemented the architecture in RIOT OS [‡] and evaluated it on the FIT IoT-LAB [§] testbed under the three control-plane deployments. In the distributed deployment, we decompose RPL into 11 VFs and enable runtime switching between storing and non-storing modes. In the centralized deployment, the orchestrator operates as an SDN-like controller and programs nodes via VFs carrying rules. The hybrid deployment combines both: nodes run the distributed RPL control plane locally, while the orchestrator restores preserved state after updates to avoid long reconvergence. VFs are implemented using Femto Containers (FCs) [Z$^+$22], and experiments adopt the IETF-recommended stack (IPv6/6LoWPAN/IEEE 802.15.4) while our architecture remains protocol-agnostic. All experiments use a 46-node topology.

To quantify update size and update time, we (i) switch RPL between storing and non-storing in the distributed deployment; (ii) push new forwarding rules in the centralized deployment to construct a topology optimized for link quality (22 nodes at one hop, 15 at two hops, and 8 at three hops); and (iii) switch RPL modes in the hybrid deployment while preserving and restoring state (convergence rules). Table 1 summarizes the update sizes. In the centralized deployment, each rule occupies 104–208 B depending on rule-grouping optimizations, resulting in a per-node size of $N \times [104, 208]$ B where N is the number of rules installed. In the distributed deployment, only 3 of the 11 RPL VFs change when switching modes, reducing the update size by 96.01% and 96.98% compared to native full-image updates. In the hybrid deployment, the update includes the same 3 VFs plus a State VF carrying per-node rules. Across all deployments, our architecture keeps the total update time below 2,200 s, while native full-image updates exceed 27,000 s.

We assess post-update convergence and maintenance under dynamic conditions. The convergence time is defined as the time until a node becomes connected and accessible from the root node. Fig. 3a (20 runs) shows that the **distributed** deployment experiences long post-update reconvergence because nodes

‡. https://github.com/RIOT-OS
§. https://www.iot-lab.info/



(a) CDF of Node Convergence Times After Update

(b) Maintenance Recovery Time

Figure 3 – Comparing the performance of different control-plane deployments

must re-exchange control messages to rebuild routing state after mode changes. The **centralized** deployment converges nearly instantly once rules are delivered, since nodes apply controller-pushed state without neighborhood exchanges. The **hybrid** deployment achieves near-instant convergence by switching RPL modes and restoring forwarding state via the orchestrator, eliminating the reconvergence phase.

To highlight the vulnerability of centralized control under lossy, multihop, and overloaded conditions, we remove six selected children of the root node (each with at least two children, and each child with at least one descendant) and measure the time for their descendants to reconnect. We also inject application-like traffic by having the nodes transmit 1-byte UDP packets every 1 s and 0.5 s to the root node. Fig. 3b (20 runs per scenario) shows that recovery time increases with load for all deployments. Centralized control suffers the greatest recovery delays because it must collect information, recompute rules, and reliably deliver them over lossy multihop paths. Distributed control recovers faster because nodes locally discover alternative parents without relying on remote rule delivery. The hybrid deployment combines both benefits: nodes rapidly reconnect using the local control plane under adverse conditions, and the orchestrator can subsequently refine the topology using its global view.

In general, centralized deployments offer immediate convergence after rule delivery, but are sensitive to unreliable backhaul and congestion, distributed deployments avoid remote rule dissemination but incur reconvergence after updates, and hybrid deployments mitigate both by combining local autonomy with state-aware global assistance.

5 Conclusion and Future Work

Our architecture enables semantic, fine-grained runtime reconfiguration in LLWNs by decomposing the network stack into lightweight Virtual Functions (VFs) that can be updated independently, significantly reducing update size and update time. We also propose a hybrid control-plane deployment that combines the advantages of centralized approaches, providing globally optimized network decisions, with those of distributed approaches, which enable rapid reactions to local changes. By preserving network state during updates, this hybrid mode avoids costly reconvergence and improves network availability. Future work will extend validation beyond routing and integrate our architecture with digital twins to predict network conditions and validate updates before deployment, thus enhancing safety and reliability.

References

[B+18] Michael Baddeley et al. Evolving sdn for low-power iot networks. In *2018 4th IEEE NetSoft*. IEEE, 2018.

[GK12] Olfa Gaddour and Anis Koubâa. Rpl in a nutshell: A survey. *Computer Networks*, 56(14):3163–3178, 2012.

[I+20] Timofei Istomin et al. Route or Flood? Reliable and Efficient Support for Downward Traffic in RPL. *ACM Transactions on Sensor Networks*, 16(1), 2020.

[Z+22] Koen Zandberg et al. Femto-containers: lightweight virtualization and fault isolation for small software functions on low-power iot microcontrollers. In *23rd ACM/IFIP International Middleware Conference*, 2022.